avec des ptits dinos mignons

le Monde des Heaposaures

par Narcisse, fils de Lowengeist & V0odOo

C'est quoi la heap ?

En bref, une interface pour :

- **allouer** des bouts de mémoire à taille variable,
- les **supprimer** quand on s'en sert plus,

Le tout, en voulant avoir des **primitives les plus rapides possibles**, tout en faisant **attention à la sécurité de la zone** mémoire.

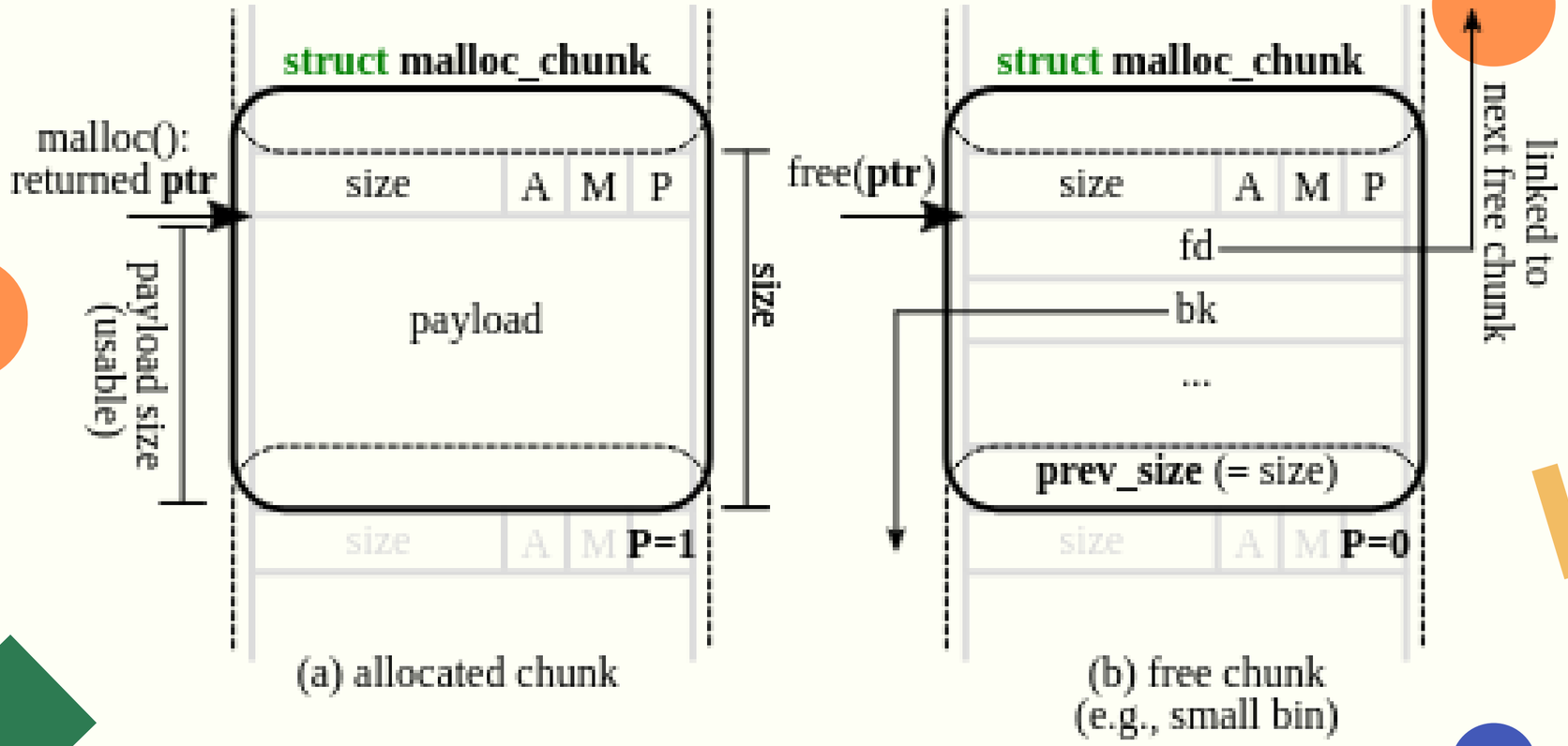
```
Start Addr          End Addr            Size                Offset
Perms File
0x0000555555400000 0x0000555555402000 0x2000              0x0
r-xp  /home/narcisse/Desktop/HackademINT/formations/heap-vanilla/data/chall
0x0000555555601000 0x0000555555602000 0x1000              0x1000
r--p  /home/narcisse/Desktop/HackademINT/formations/heap-vanilla/data/chall
0x0000555555602000 0x0000555555603000 0x1000              0x2000
rw-p  /home/narcisse/Desktop/HackademINT/formations/heap-vanilla/data/chall
0x0000555555603000 0x0000555555624000 0x21000             0x0
rw-p  [heap]
0x00007ffff7800000 0x00007ffff79ab000 0x1ab000            0x0
r-xp  /home/narcisse/Desktop/HackademINT/formations/heap-vanilla/data/libc.s
o.6
0x00007ffff79ab000 0x00007ffff7bab000 0x200000            0x1ab000
---p  /home/narcisse/Desktop/HackademINT/formations/heap-vanilla/data/libc.s
o.6
0x00007ffff7bab000 0x00007ffff7baf000 0x4000              0x1ab000
r--p  /home/narcisse/Desktop/HackademINT/formations/heap-vanilla/data/libc.s
```

Et comment ça marche ?

Explication **par le code** ici : **https://elixir.bootlin.com**
Pour ceux qui ont la flemme, en pratique c'est différentes wheels correspondant à différentes tailles :

- **tcache** (des petits chunks auxquels on accède rapidement),
- fastbin (toujours des petits chunks quand il y a plus de place dans la tcache wheel correspondante),
- largebin (des chunks plus gros),
- **unsortedbin** (des chunks de taille variable qui ont récemment été free, ici pour des raisons de vitesse),

Un schema pour tout comprendre :



Le point GDB

```
gef➢  heap chunk 0x55555555ae00
Chunk(addr=0x55555555ae00, size=0x20, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)
Chunk size: 32 (0x20)
Usable size: 24 (0x18)
Previous chunk size: 0 (0x0)
PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA
```

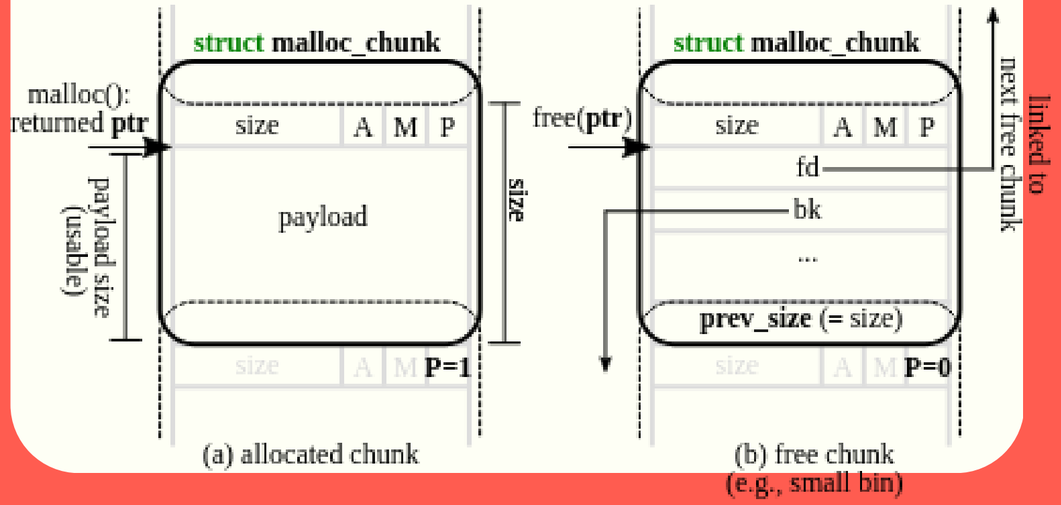
```
➢  heap bins
──────────────────────────────────────────────────────────────────────────────── Tcachebins for thread 1 ────────────────────────────────────────────────────────────────────────────────
chebins[idx=0, size=0x20, count=7] ←  Chunk(addr=0x55555555c0d0, size=0x20, flags=PREV_INUSE | IS_MMAPP
r=0x55555555ae00, size=0x20, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)  ←  Chunk(addr=0x55555555bc
| IS_MMAPPED | NON_MAIN_ARENA)  ←  Chunk(addr=0x55555555bae0, size=0x20, flags=PREV_INUSE | IS_MMAPPED | NON
0x55555555b940, size=0x20, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)  ←  Chunk(addr=0x55555555b7a0, size=0x20, flag
IS_MMAPPED | NON_MAIN_ARENA)  ←  Chunk(addr=0x55555555ad70, size=0x20, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)
chebins[idx=5, size=0x70, count=7] ←  Chunk(addr=0x55555555b7e0, size=0x70, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)
dr=0x55555555b980, size=0x70, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)  ←  Chunk(addr=0x55555555bb20, size=0x70, flag
E | IS_MMAPPED | NON_MAIN_ARENA)  ←  Chunk(addr=0x55555555bcc0, size=0x70, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)  ←
dr=0x55555555be60, size=0x70, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)  ←  Chunk(addr=0x55555555bfe0, size=0x70, flags=
E | IS_MMAPPED | NON_MAIN_ARENA)  ←  Chunk(addr=0x55555555ad90, size=0x70, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)
Tcachebins[idx=6, size=0x80, count=7] ←  Chunk(addr=0x55555555b720, size=0x80, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_AREN
(addr=0x55555555b8c0, size=0x80, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)  ←  Chunk(addr=0x55555555ba60, size=0x80,
USE | IS_MMAPPED | NON_MAIN_ARENA)  ←  Chunk(addr=0x55555555bc00, size=0x80, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_AR
dr=0x55555555bda0, size=0x80, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)  ←  Chunk(addr=0x55555555c050, size=0x80,
| IS_MMAPPED | NON_MAIN_ARENA)  ←  Chunk(addr=0x55555555acf0, size=0x80, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)
```

Le tcachus poisonneus

ECRIRE OÙ ON VEUT

L'attaque de heap moderne a la base de TOUT
On émet l'hypothese qu'on a une primitive d'ecriture sur un chunk qui est dans le tcache {petit chunk + freed},

- On va overwrite les pointeurs de la wheel avec une addresse qu'on veut,
- Quand le tcache voudra nous donner un ptit chunk, il nous donnera un pointeur sur une adresse arbitraire!



Le tcachus poisonneus

ECRIRE OÙ ON VEUT

Attention a 1 truc :

- la consolidation : des chunks recemment free qui sont colles seront fusionnes pour eviter de se retrouver avec une heap qui est fragmentee (pleine de ptits grumeaux)
- pour eviter Ça, en general on met des chunks entre nos chunks a free pour eviter toute consolidation non voulue

le leakosorus

TROUVER OÙ ÉCRIRE

pour trouver des leaks, on a en general besoin d'avoir une primitive en lecture sur un chunk desalloue.

Selon la wheel on obtiendra :

- une adresse de la libc si le chunk est assez gros {soit > 1M => mmap threshold, soit ~0x400 pour de l'unsorted},
- une adresse de la heap si le chunk est petit,

dans les deux cas, c'est interessant : un leak heap peut aider a construire un leak libc, un leak libc permet de réécrire FREE HOOK

L'instant __free_hook

Comment transformer une primitive d'écriture sur la libc en shell ?

- les vieilles versions de la libc (<2.34, vérifiez avec strings libc.so.6 | grep GNU) contenaient des fonctions __free_hook et __malloc_hook permettant de définir des routines modifiant l'exécution de malloc et free,
- la zone où ces fonctions sont définies est writeable, il suffit donc d'y placer une adresse de fonction qu'on aime bien et c'est gagné ! (par exemple system 😉)

Construire un exploit a partir de tout ça :

star.hackademint.org => heap-vanilla